The Python Standard Library: Essential Modules

1. Filesystem Operations: os and pathlib

The `os` module provides traditional operating system dependent functionality, such as `os.getcwd()` and `os.listdir()`. However, modern Python development heavily favors the object-oriented `pathlib` module introduced in PEP 355.

`pathlib.Path` represents file system paths with semantics appropriate for different operating systems. It handles path joining using the `/` operator, globbing, and file reading with clean, readable syntax.

```
from pathlib import Path base_dir = Path('/var/log') log_files =
list(base_dir.glob('*.log')) for log in log_files: print(log.name,
log.stat().st_size)
```

2. Data Serialization: json and csv

Serialization allows Python objects to be converted into data streams for storage or transmission. The `json` module supports encoding and decoding JSON data via `json.dumps()` and `json.loads()`.

For tabular data, the `csv` module provides robust parsing capabilities. `csv.DictReader` parses CSV rows directly into ordered dictionaries, mapping column headers automatically to row values.

```
import json, csv data = {'status': 'success', 'codes': [200, 201]} json_str =
json.dumps(data, indent=2)
```

3. Date and Time Handling: datetime

The `datetime` module supplies classes for manipulating dates and times. A crucial distinction exists between naive datetimes (which lack timezone context and assume local time) and aware datetimes (which include offset information relative to UTC).

Best practices dictate storing and processing all timestamps in UTC using `datetime.timezone.utc` and converting to local time zones only at the presentation layer.

```
from datetime import datetime, timezone, timedelta now_utc =
datetime.now(timezone.utc) future_time = now_utc + timedelta(days=7)
```

4. Specialized Containers: collections

The `collections` module implements specialized container datatypes providing alternatives to Python's general purpose built-in containers:

• `namedtuple`: Tuple subclasses with named fields for readable, lightweight records.

• `deque`: Double-ended queue supporting fast O(1) appends and pops from both ends.

• `Counter`: Dict subclass for counting hashable objects.

• `defaultdict`: Dict subclass that calls a factory function to supply missing values.

```
from collections import Counter, defaultdict word_counts = Counter(['apple',
'banana', 'apple', 'orange'])
```

5. Regular Expressions: re

The `re` module provides regular expression matching operations similar to Perl. Using raw strings (`r'pattern'`) is essential to prevent unintended backslash escapes in patterns.

Key functions include `re.search()`, `re.match()`, `re.findall()`, and `re.sub()` for pattern replacement.

```
import re text = 'Contact support@example.com for help.' email =
re.search(r'[\w\.-]+@[\w\.-]+', text).group(0)
```